

Sayısal Değişkenler ve Matematiksel Operatörler

Görsel Programlama - Hafta 5

HAFTA 5: Sayısal Değişkenler ve Matematiksel Operatörler

Bu ders, Bilişim Güvenliği Teknolojisi programı kapsamındaki Görsel Programlama dersinin beşinci haftasını kapsamaktadır. Temel amacımız, bilgisayarların sayısal veriyi nasıl depoladığını ve matematiksel işlemleri nasıl gerçekleştirdiğini anlamaktır. Sayısal veri tiplerinin doğru kullanımı, yazılım performansını ve veri doğruluğunu doğrudan etkiler.



Dersin Amaçları ve Öğrenim Hedefleri

Öğrenciler bu haftanın sonunda:

1. Temel sayısal veri tiplerini (int, double) tanımlayabileceklerdir.
2. Dört temel aritmetik operatörü (+, -, *, /) programlama bağlamında kullanabileceklerdir.
3. İşlem önceliği kurallarını uygulayarak karmaşık matematiksel ifadeleri çözümleyebileceklerdir.
4. Bölme işleminde tam sayı ve ondalıklı sonuç farkını yönetebileceklerdir.



Bilgisayarda Veri Tipleri Neden Önemlidir?

Veri tipleri, derleyicinin bellekte ne kadar alan ayıracağını (örneğin 4 bayt veya 8 bayt) ve bu alandaki bitleri nasıl yorumlayacağını belirler.

Doğru tip seçimi, belleği verimli kullanmamızı sağlar (Kaynak Kısıtlaması).
Hatalı tip seçimi (örneğin çok büyük bir sayıyı int olarak saklamaya çalışmak),
'taşma' (overflow) hatalarına yol açar.

Programlama dilleri (özellikle C#), veri tiplerini katı bir şekilde kontrol eden
'güçlü tipli' (strongly-typed) dillerdir.



Sayısal Veri Tiplerine Giriş

Programlamada sayılar genellikle iki ana kategoriye ayrılır:

1. Tam Sayılar (Integers): Kesirli kısmı olmayan sayılar.
2. Ondalıklı Sayılar (Floating-Point Numbers): Kesirli veya çok hassas ölçüm gerektiren sayılar.

C# ve birçok modern dilde bu kategoriler için farklı tipler tanımlanmıştır: int, double, float, decimal.



Tam Sayılar (Integer - int)

int, İngilizce 'Integer' kelimesinin kısaltmasıdır ve tam sayıları saklamak için kullanılır.

Pozitif, negatif veya sıfır olabilirler, ancak kesirli kısımları yoktur.

C#'ta int tipi genellikle 32-bit (4 bayt) alan kaplar.

Kullanım Alanları: Yaş, öğrenci sayısı, stok adedi, döngü sayaçları gibi kesirsiz hesaplamalar.



int Veri Tipinin Sınırları ve Taşma (Overflow)

int tipi, yaklaşık olarak $-2.147.483.648$ ile $+2.147.483.647$ arasındaki sayıları saklayabilir.

Eğer bir hesaplama bu sınırı aşarsa, 'tam sayı taşması' (integer overflow) meydana gelir.

Taşma, özellikle güvenlik açıklarına ve beklenmedik program davranışlarına neden olabilir (Örn: Yıl 2038 sorunu).

Büyük tam sayılar gerektiğinde, C#'ta 'long' (64-bit) gibi alternatif tipler kullanılmalıdır.

int Kullanım Örneği

int degiskenAdi = Deger; şeklinde tanımlanır.

Örnekler:

int yas = 25; // Bir kişinin yaşı

int ogrenciSayisi = 120; // Sınıftaki öğrenci sayısı

int sicaklik = -5; // Sıcaklık derecesi (negatif tam sayı)

Değişken isimleri (yas, ogrenciSayisi) anlamlı ve büyük/küçük harf duyarlı olmalıdır.



Ondalıklı Sayılar (Floating-Point Numbers)

double, çift hassasiyetli kayan noktalı sayıları (Double Precision Floating Point) saklamak için kullanılır.

Hassas ölçümler, finansal değerler, bilimsel hesaplamalar ve not ortalamaları için idealdir.

C#'ta double tipi 64-bit (8 bayt) bellek kullanır ve çok geniş bir aralığı ve yüksek hassasiyeti kapsar.

Örnek: double fiyat = 19.99;

C#'ta ondalık ayırıcı olarak nokta (.) kullanılır, virgöl (,) değil.

Ondalıklı Sayılar: float ve double Karşılaştırması

float (tek hassasiyetli) 32-bit kullanır ve yaklaşık 7 basamak hassasiyet sunar.
double (çift hassasiyetli) 64-bit kullanır ve yaklaşık 15-17 basamak hassasiyet sunar.

Günümüz modern programlamasında, özellikle C#'ta, ondalıklı işlemler için double varsayılan ve tercih edilen tiptir, çünkü daha az hassasiyet kaybı yaşanır.

float tipi, büyük miktarda grafik verisi veya bellek kısıtlı cihazlar için kullanılabilir.

IEEE 754 Standardı

Tüm modern bilgisayar sistemleri, kayan noktalı sayıları temsil etmek için IEEE 754 standardını kullanır.

Bu standart, sayıları (işaret), (üssün) ve (mantis) bileşenlerine ayırarak depolar.

Bu temsil biçimi, çok geniş sayı aralıklarını depolamaya izin verir, ancak bazen tam olarak temsil edilemeyen ondalık değerler (örneğin 0.1) nedeniyle küçük yuvarlama hatalarına yol açabilir.



Sayısal Değişkenlerin Tanımlanması

Değişken tanımlaması: Veri_Tipi değişken_Adı = Başlangıç_Değeri;

int stokAdedi = 350;

double urunFiyati = 49.90;

double KDV_Orani = 0.20;

int ayarlanabilirHiz = 80;

Bir değişkeni tanımladıktan sonra, programın akışı sırasında değerini değiştirebiliriz (örneğin `stokAdedi = stokAdedi - 1;`).

Finansal Hesaplamalar İçin Özel Tip: decimal

double, yüksek hızda bilimsel hesaplamalar için optimize edilmiştir ancak IEEE 754 standardından gelen küçük hassasiyet hataları finansal işlemlerde kabul edilemez.

Finansal ve para birimi hesaplamaları için C#'ta 'decimal' tipi kullanılır. decimal, 128-bit kullanır ve temel 10 tabanında (onluk sistemde) yüksek hassasiyetle çalışır.

Örnek: decimal maaş = 5500.75m; (sonundaki 'm' harfi decimal olduğunu belirtir).

Değişkenler ve Bellek İlişkisi

Bir değişken tanımlandığında, bilgisayar belleğinde o veri tipine uygun bir alan ayrılır.

Değişken Adı: Bellekteki bu alana erişmek için kullanılan etikettir.

Değer: Bellekte saklanan gerçek bilgidir (örneğin 25).

Aritmetik Operatörler: Giriş

Programlamada, matematiksel hesaplamaları gerçekleştirmek için aritmetik operatörler kullanılır.

Bu operatörler, bir veya daha fazla sayısal değeri alıp yeni bir sayısal değer üretirler (operandlar ve sonuç).

Temel operatörler şunlardır:

- + (Toplama)
- (Çıkarma)
- * (Çarpma)
- / (Bölme)



Toplama Operatörü (+)

İki sayısal değeri toplamak için kullanılır.

```
int toplam = sayi1 + sayi2;
```

Eğer farklı sayısal tipleri toplarsanız (örneğin int + double), C# otomatik olarak sonucu daha geniş tipe (bu durumda double) yükseltir (Implicit Conversion).



Çıkarma Operatörü (-)

Bir sayıdan diğerini çıkarmak için kullanılır.

```
int fark = sayiA - sayiB;
```

Çıkarma işlemi sonucu pozitif, negatif veya sıfır olabilir.

Bu operatör ayrıca bir sayının işaretini değiştirmek için de kullanılabilir (Unary Negation):

```
int negatifSayi = -pozitifSayi;
```



Çarpma Operatörü (*)

İki sayıyı çarpmak için kullanılır.

double alan = en * boy;

Çarpma işlemi, toplama/çıkarma işlemlerinden önce gelir (İşlem Önceliği).

Bölme Operatörü (/)

Bir sayıyı diğerine bölmek için kullanılır.

Bölme operatörünün davranışı, işleme giren sayıların (operandların) veri tipine bağlıdır.

Eğer her iki operand da double ise, sonuç ondalıklı olacaktır (2.5 gibi).

Eğer her iki operand da int ise, sonuç da int olacaktır ve ondalıklı kısım atılacaktır (truncation).



Mod Alma Operatörü (%)

Mod alma operatörü (%), bir bölme işleminden sonra kalan tam sayıyı bulmak için kullanılır (C# kaynak metninde belirtilmemiştir ancak temel aritmetik operatörler arasındadır ve üniversite düzeyinde bilinmelidir).

Örnek: $10 \% 3$ işleminin sonucu 1'dir (10'u 3'e böldüğümüzde kalan 1'dir).

Kullanım Alanları: Bir sayının çift mi tek mi olduğunu kontrol etme, döngüleri belirli aralıklarla çalıştırma, zaman hesaplamaları (dakika/saniye).

Operatörlerin Değişkenlerle Kullanımı

Matematiksel işlemler genellikle, bir sonucun bir değişkene atanmasıyla gerçekleştirilir.

Syntax: VeriTipi sonuc = ifade;

Örnek: int sayi1 = 20; int sayi2 = 5;

int sonuc = sayi1 - sayi2; // sonuc = 15

Bu örnekte, sayi1 ve sayi2 değerleri okunur, işlem yapılır ve çıkan sonuç (15) 'sonuc' adlı int değişkene yazılır.

İşlem Önceliği (Operator Precedence)

Birden fazla operatör içeren ifadelerde, işlemlerin hangi sırayla yapılacağını belirleyen kurallar bütünüdür.

Programlamada (ve matematikte) belirli bir öncelik sırası geçerlidir:

1. Parantezler (en yüksek öncelik)
2. Çarpma (*), Bölme (/), Mod (%) (ikinci öncelik)
3. Toplama (+), Çıkarma (-) (en düşük öncelik)

İşlem Önceliği Uygulaması: Örnek 1

İfade: $\text{int sonuc} = 5 + 3 * 2;$

1. Öncelikle Çarpma yapılır: $3 * 2 = 6$

2. Ardından Toplama yapılır: $5 + 6 = 11$

Sonuç: sonuc değişkeninin değeri 11 olur.

Eğer öncelik kuralı uygulanmasaydı (soldan sağa), sonuç 16 olabilirdi ($5+3=8$, $8*2=16$), ki bu yanlıştır.

Parantezlerin Gücü

Parantezler, matematiksel ifade içinde, parantez içindeki işlemin öncelikli olarak yapılmasını sağlar.

İfade: $\text{int sonuc} = (5 + 3) * 2;$

1. Öncelikle Parantez içi yapılır: $5 + 3 = 8$

2. Ardından Çarpma yapılır: $8 * 2 = 16$

Parantez kullanarak, doğal işlem önceliğini geçersiz kılar ve programcının istediği hesaplama sırasını zorunlu kılabiliriz.

Aynı Öncelikli Operatörler: Bağdaştırıcılık (Associativity)

Eğer bir ifadede aynı öncelik seviyesindeki operatörler varsa (örneğin * ve /, veya + ve -), işlem sırası soldan sağa doğru belirlenir (sol bağdaştırıcılık).

Örnek: $10 / 2 * 5$

1. Önce soldaki bölme: $10 / 2 = 5$

2. Ardından çarpma: $5 * 5 = 25$

Eğer sağdan sola yapılsaydı ($2*5=10$, $10/10=1$) sonuç farklı olurdu. Bu kural, tutarlılık sağlar.

Tam Sayı Bölmesi Sorunu (Integer Division)

Programlamada, iki tam sayı (int) birbirine bölündüğünde, sonuç her zaman yine bir tam sayı (int) olur.

Ondalıklı kısım, yuvarlanmaz; direkt olarak ATILIR (truncation - kesme).

Bu, birçok yeni programcının yaptığı yaygın bir hatadır ve beklenmedik sonuçlara yol açar.

Örnek: `int sonuc = 10 / 4;` Sonuç 2 olur, 2.5 değil.

Tam Sayı Bölmesinden Doğru Sonuç Alma

Ondalıklı (double) sonuç elde etmek için, bölme işlemindeki operandlardan en az birinin ondalıklı tipte olması gerekir.

Bu durumda C#, 'implicit conversion' (örtülü dönüşüm) yaparak diğer operandı da double'a yükseltir ve kayan noktalı bölme gerçekleştirir.

Doğru Örnek: `double sonuc = 10.0 / 4; // sonuc 2.5 olur`

Diğer Doğru Örnek: `double sonuc = 10 / 4.0; // sonuc 2.5 olur`

Tipi Zorlama (Casting)

Eğer bölme işlemindeki değişkenler int olarak tanımlıysa, sonucu double'a dönüştürmek için 'explicit casting' (açık tip zorlama) kullanılmalıdır.

Örnek:

```
int a = 10; int b = 4;
```

```
double sonuc = (double)a / b;
```

'(double)a' ifadesi, 'a' değişkeninin değerini geçici olarak double tipine dönüştürür. Bu, bölme işlemini kayan noktalı hale getirir ve doğru sonucu (2.5) elde ederiz.

İşlem Sonuçlarının Karıştırılması

C#, farklı sayısal tipleri içeren işlemlerde, sonucu her zaman daha 'güvenli' ve daha 'geniş' tipte tutmaya çalışır.

$\text{int} + \text{double} = \text{double}$

$\text{float} * \text{double} = \text{double}$

$\text{int} / \text{double} = \text{double}$

Bu otomatik yükseltme (promotion), hassasiyet kaybını önlemeyi amaçlar, ancak programcının sonucu nereye atadığı önemlidir.

Sayısal Değişkenler ve Atama Operatörleri

Bir değişkenin değerini kendisiyle işlem yaparak güncellemek yaygın bir senaryodur (örneğin sayacı 1 artırmak).

Klasik Atama: `sayi = sayi + 1;`

Kısa Yol Atama Operatörleri:

`+=` (Topla ve Ata): `sayi += 1;` // `sayi = sayi + 1` ile aynıdır.

`-=` (Çıkar ve Ata): `sayi -= 5;` // `sayi = sayi - 5` ile aynıdır.

`*=` (Çarp ve Ata)

`/=` (Böl ve Ata)

Bunlar kodu kısaltır ve okunabilirliği artırır.

Artırma ve Eksiltme Operatörleri (Increment/Decrement)

Özellikle döngülerde veya sayım işlemlerinde bir değişkenin değerini 1 artırmak (++) veya 1 azaltmak (--) için kullanılırlar.

++ (Increment Operatörü): `sayi++`; veya `++sayi`;

-- (Decrement Operatörü): `sayi--`; veya `--sayi`;

Ön (Prefix) ve Son (Postfix) Kullanım Farkı: `++sayi` (işlem önce yapılır) ve `sayi++` (işlem sonra yapılır) farklı sonuçlar üretebilir, bu durum özellikle karmaşık ifadelerde önemlidir.

Uygulama Alanı: Basit Hesap Makinesi Projesi

Bu hafta öğrendiğimiz sayısal veri tipleri ve operatörler, basit bir 4 işlem hesap makinesi uygulamak için mükemmel bir temel oluşturur.

Proje Hedefi: Kullanıcıdan iki sayı alıp, seçilen operatöre göre sonucu ekranda göstermek.

İhtiyaç duyulan bileşenler: İki adet sayı girişi için TextBox, sonucu göstermek için bir Label, ve dört işlem için dört ayrı Button.



Hesap Makinesi Arayüz Tasarımı

Kullanılacak Kontroller:

1. Sayı 1 Girişi: txtSayi1 (TextBox)
 2. Sayı 2 Girişi: txtSayi2 (TextBox)
 3. Sonuç Çıktısı: lblSonuc (Label)
 4. İşlem Butonları: btnTopla, btnCikar, btnCarp, btnBol (Button)
- Label'lar (Sayı 1:, Sayı 2:, Sonuç:) kullanıcıya yol göstermek için kullanılır.

Kodlama Öncesi Kavramsal Sorun: Veri Dönüşümü

TextBox'lar, kullanıcıdan alınan veriyi her zaman METİN (string) olarak saklar. Fakat matematiksel operatörler (+, -, *, /) sadece SAYISAL tipler (int, double) üzerinde çalışır.

Bu aşamada (sonraki haftalarda göreceğiz), TextBox'tan okunan metnin sayıya (double veya int) dönüştürülmesi gerekir (Parsing).



Örnek Kod Yapısı: Toplama İşlemi

Bu haftaki örnekte, veri dönüşümünü basitleştirmek için sayıları doğrudan kod içine manuel olarak double tanımladık.

Bu hafta: `double sayi1 = 10.5; double sayi2 = 5.2;`

İşlem: `double toplam = sayi1 + sayi2;`

Sonuç Görüntüleme: `lblSonuc.Text = toplam.ToString();`

Sayısal Sonucu Görsel Nesneye Yazdırma: .ToString()

lblSonuc.Text gibi metin tabanlı kontroller (Label, TextBox, ListBox vb.) yalnızca metin (string) verisi kabul ederler.

Programlama dillerinde, sayısal bir değeri bu kontrollere atamadan önce metin tipine dönüştürmek ZORUNLUDUR.

Bu dönüşüm, C#'ta .ToString() metodu ile gerçekleştirilir.

Bu, veri tipleri arasındaki katı ayrımın ve dönüşüm ihtiyacının ilk işaretidir.

Diğer Aritmetik İşlemlerin Uygulanması

btnCikar, btnCarp ve btnBol butonları için de benzer kod yapısı tekrarlanır. Bu tekrar, öğrencilerin operatörlerin kullanımını pekiştirmesi için önemlidir. Tek değişen kısım kullanılan aritmetik operatördür ($-$, $*$, $/$).

Haftaya Hazırlık: Kullanıcıdan Girdi Alma

Gelecek hafta, hesap makinesi projemizi gerçek kullanıcı girdisi ile çalışacak şekilde güncelleyeceğiz.

Bu, TextBox'tan metin (string) okuma ve bu metni sayısal tipe (int veya double) dönüştürme (Parse/Convert) becerilerini gerektirecektir.

Bu dönüşümler, güvenli programlama için kritik öneme sahiptir.



Akademik Genişleme: Tip Güvenliği

C#, Java gibi diller statik tip güvenliğine sahiptir. Bu, derleme zamanında tip hatalarını yakalamak anlamına gelir.

Tip Güvenliği:

1. Güvenilirlik: Programın beklenmedik tip dönüşümlerinden dolayı çökmesini veya yanlış hesaplama yapmasını engeller.
2. Performans: Derleyici, tipleri bildiği için daha optimize edilmiş makine kodu üretebilir.
3. Okunabilirlik: Kodun hangi veriyi tuttuğu açıkça belirtilmiştir (int, double).

Özet ve Temel Kavramlar

int: Tam sayılar için 32-bit tip.

double: Ondalıklı sayılar için 64-bit tip.

İşlem Önceliği: *, / önce, +, - sonra gelir; parantezler en önceliklidir.

Integer Bölme: İki int'in bölümü, ondalıklı kısmı keser (truncation).

Veri Tipi Dönüşümü: Sayısal sonucu metin tabanlı kontrole yazmak için .ToString() kullanılır.

Ekstra Detay: Unary Operatörler

Tek bir operand üzerinde çalışan operatörlerdir:

Unary Artı (+): Sayının pozitif olduğunu belirtir (genellikle ihmal edilir).

Unary Eksi (-): Sayının işaretini tersine çevirir.

Örnek: `int a = 5; int b = -a; // b artık -5'tir.`

`++` ve `--` operatörleri de (artırma ve eksiltme) tek operand aldıkları için unary operatörlerdir.

Akademik Genişleme: Karşılaştırma Operatörleri

Sayısal veriler üzerinde sadece hesaplama değil, karşılaştırma da yaparız. Bu operatörler, sonuç olarak bir boolean (True/False) değeri döndürür.

== (Eşittir)

!= (Eşit Değildir)

> (Büyüktür)

< (Küçüktür)

>= (Büyük Eşittir)

<= (Küçük Eşittir)

Bunlar, programın akışını kontrol eden (if, while) koşullu ifadelerin temelini oluşturur.



Mantıksal Operatörler ve Sayısal Koşullar

Karşılaştırma operatörleri genellikle mantıksal operatörlerle birleştirilir:

- && (Mantıksal VE / AND): Her iki koşul da doğruysa True.
- || (Mantıksal VEYA / OR): Koşullardan en az biri doğruysa True.
- ! (Mantıksal DEĞİL / NOT): Koşulun tersini alır.

Örnek: `(yas > 18) && (krediSkoru >= 700)`

Deneyisel Hesaplama: Kesme Hataları

double veya float kullanırken, 0.1 gibi ondalıklı sayıların ikili sistemde (binary) tam olarak temsil edilememesinden kaynaklanan hatalar oluşabilir.
Örnek: $0.1 + 0.2$ işleminin sonucu tam olarak 0.3 olmayabilir (0.30000000000000004 gibi). Bu hassasiyet kaybı 'kesme hatası' olarak bilinir. Bu nedenle eşitlik kontrolü ($==$) yerine genellikle küçük bir tolerans (epsilon) değeri ile karşılaştırma yapılır.

Sayısal Veri Tipi Seçim Stratejileri

1. Sayı tam sayı mı? Evetse int kullan (yeterli aralıkta ise).
2. Sayı ondalıklı mı?
 - a. Hassasiyet kritik mi (Para, finans)? decimal kullan.
 - b. Bilimsel/geometrik hesaplama mı? double kullan.



Kod Kalitesi: Değişken Adlandırma Kuralları

C#'ta değişken adlandırması için yaygın olarak Camel Case kullanılır: ilk kelime küçük harfle başlar, sonraki kelimelerin ilk harfi büyük olur (örneğin `urunFiyati`, `ogrenciSayisi`).

Değişken adları, sakladıkları verinin amacını yansıtmalıdır (self-documenting code).

Türkçe karakterler (ğ, ş, ı, ö, ç) değişken adlarında kullanılmamalıdır.

Sabitler (Constants) Kavramı

Bazı sayısal değerler (örneğin Pi sayısı, KDV oranı) program boyunca asla değişmemelidir.

Bu tür değerler 'const' anahtar kelimesi kullanılarak sabit olarak tanımlanır.

Örnek: `const double PI = 3.14159;`

Sabit kullanmak, kodun okunabilirliğini artırır ve yanlışlıkla değiştirilmesini engeller.



Uygulama İpuçları: Kullanıcı Deneyimi

Bir TextBox'tan sayı bekliyorsak, kullanıcının metin (string) girmesini engellemeliyiz (Gelecek haftaların konusu).
Sayısal bir değer girilmezse ve biz onu dönüştürmeye çalışırsak (parsing), program 'format hatası' (FormatException) verecektir.
Güvenilir uygulamalar, bu tür hataları TryParse gibi yöntemlerle önceden yakalar.

Bitwise Operatörler (Kısa Tanıtım)

Üniversite düzeyinde bilinmesi gereken, sayısal verinin bit seviyesinde manipülasyonunu sağlayan operatörlerdir.

& (AND), | (OR), ^ (XOR), ~ (NOT), << (Left Shift), >> (Right Shift).

Kullanım Alanları: Düşük seviyeli donanım kontrolü, ağ protokolleri, kriptografi ve güvenlik uygulamaları (dersin bağlamına uygundur).



Kriptografide Sayısal Operatörler

Kriptografik algoritmaların büyük çoğunluğu, çok büyük tam sayılar üzerinde hızlı ve doğru aritmetik işlemlere dayanır.

Örneğin, RSA gibi açık anahtarlı şifreleme sistemleri, mod alma (%) ve üs alma işlemlerini yoğun olarak kullanır.

Integer overflow gibi hatalar, kriptografik anahtarların kırılmasına yol açabilecek güvenlik zafiyetleri oluşturabilir.



Quiz Sorusu 1: Temel Tip Bilgisi

Aşağıdaki değerlerden hangileri int ve hangileri double olarak tanımlanmalıdır?

1. Bir ülkenin nüfusu (int)
2. Bir banka hesabındaki bakiye (double veya decimal)
3. Bir cismin yerçekimi ivmesi (double)
4. Bir otobüsteki koltuk sayısı (int)
5. Ders notu ortalaması (double)

Quiz Sorusu 2: İşlem Önceliği

Aşağıdaki C# ifadesinin sonucu nedir?

```
int result = 10 - 2 * (3 + 1) / 4;
```

Çözüm:

1. Parantez içi: $(3 + 1) = 4$
2. Çarpma/Bölme (Soldan sağa): $2 * 4 = 8$
3. Devam eden Bölme: $8 / 4 = 2$
4. Çıkarma: $10 - 2 = 8$

Doğru Sonuç: 8

Quiz Sorusu 3: Integer Division

Aşağıdaki iki ifadenin sonuçları ne olur?

İfade A: `int x = 15 / 6; // x = ?`

İfade B: `double y = 15.0 / 6; // y = ?`

Cevaplar:

İfade A'da int bölme olduğu için 2 olur (ondalık kısım atılır).

İfade B'de ondalıklı bölme olduğu için 2.5 olur.

Örnek Proje Adımı 1: Tasarım ve Tanımlama

Hesap makinesi için kodlamaya başlarken, ilk adım girdi ve çıktı değişkenlerini tanımlamaktır.

Değişkenler double olarak tanımlanmalıdır, çünkü kullanıcı ondalıklı sayılar girebilir (fiyat, ortalama vb.).

Bu hafta için: double sayi1, sayi2, sonuc;

Bu değişkenler, butonlara basıldığında TextBox'lardan alınan değerleri tutacaktır.



Örnek Proje Adımı 2: Toplama Fonksiyonu

```
private void btnTopla_Click(object sender, EventArgs e)
{
    // Geçici manuel değerler
    double sayi1 = 10.5;
    double sayi2 = 5.2;
    double toplam = sayi1 + sayi2;
    lblSonuc.Text = toplam.ToString(); // Sonucu metne çevirme
}
```

Örnek Proje Adımı 3: Çıkarma ve Çarpma

Diğer butonlar için sadece operatör değiştirilir:

btnCikar: $\text{double fark} = \text{sayi1} - \text{sayi2};$

btnCarp: $\text{double carpim} = \text{sayi1} * \text{sayi2};$

Her işlem sonrasında `.ToString()` metodu kullanılarak sayısal sonucun ekranda düzgün gösterilmesi sağlanır.

Örnek Proje Adımı 4: Bölme Fonksiyonu

double kullandığımız için integer division hatası almayız. Ancak sıfıra bölme hatası (Division by Zero Exception) kritiktir.

Sıfıra Bölme Kontrolü: Eğer $\text{sayi2} == 0$ ise, bölme yapmadan kullanıcıya hata mesajı verilmelidir.

Eğer sıfıra bölme gerçekleşirse, double tipi 'Infinity' veya 'NaN' (Not a Number) gibi özel değerler üretebilir.



Akademik Genişleme: Veri Tiplerinde İki Yönlü Dönüşüm

Programlamada sürekli olarak iki yönlü dönüşüm yapılır:

1. String'den Sayıya (Girdi): Kullanıcı girdisi (TextBox) metin olduğu için, hesaplama yapmadan önce Parse, TryParse veya Convert.ToDouble() ile sayıya çevrilmelidir (Sonraki hafta).
2. Sayıdan String'e (Çıktı): Hesaplanan sonucu (double) ekranda göstermeden önce .ToString() ile metne çevrilmelidir.

Formatlama Kontrolü: .ToString() Metodu

.ToString() metodu parametresiz kullanıldığında varsayılan formatı kullanır. Ancak sayıyı belirli bir formata dönüştürmek mümkündür (örneğin para birimi, yüzde veya belirli bir ondalık basamak sayısı).

Örnekler:

```
Sayi.ToString('C'); // Para birimi (₺1.000,00)
```

```
Sayi.ToString('N3'); // Sayı formatı (1,000.000)
```

Bu, görsel programlamada çıktının kalitesini artırır.

Performans ve Tip Seçimi

int (32-bit) işlemleri, double (64-bit) işlemlerine göre genellikle daha hızlıdır ve daha az bellek tüketir.

Ancak modern işlemciler 64-bit mimariye sahip olduğundan, double işlemlerinin hız farkı çok azalmıştır.

Kural: Eğer kesirli kısma ihtiyacınız yoksa, hız ve bellek verimliliği için int kullanın. Hassasiyet öncelikliyse double veya decimal kullanın.

Örnek: Ortalama Hesaplama

Diyelim ki iki int notu topluyoruz ve ortalamayı hesaplamak istiyoruz.

```
int not1 = 75; int not2 = 88;
```

```
int toplam = not1 + not2; // 163
```

Yanlış Bölme: `double ortalama = toplam / 2; // Sonuç: 81.0` (int bölme $163/2=81$)

Doğru Bölme: `double ortalama = (double)toplam / 2; // Sonuç: 81.5` (casting ile)



Hata Ayıklama (Debugging) İpuçları

Sayısal hesaplamalarda hata ayıklarken dikkat edilmesi gerekenler:

1. İşlem Önceliği: İfadenizin parantezlerle matematiksel beklentinize uygun olduğundan emin olun.
2. Tip Kontrolü: Özellikle bölme işlemlerinde (/) her iki operandın da int olup olmadığını kontrol edin. Ondalıklı sonuç bekliyorsanız en az birini double yapın.
3. Taşma Kontrolü: Tanımlanan tipin (int) alabileceği maksimum değeri aşmadığınızdan emin olun.



Kaynak Kodu Okunabilirliği

Matematiksel bir ifadenin neden belirli bir şekilde yazıldığı, özellikle karmaşık formüller kullanıldığında, yorumlarla açıklanmalıdır.

Örnek:

```
// Önce KDV'yi hesapla, sonra ana fiyata ekle
```

```
double toplamFiyat = fiyat + (fiyat * kdvOrani);
```

Yorumlar, hem programcının kendisi hem de ekip arkadaşları için kodun bakımını kolaylaştırır.



Özet ve Tekrar: C# Veri Tipleri Hiyerarşisi

C#'ta sayısal tipler, genellikle bellek boyutu ve hassasiyetlerine göre bir hiyerarşiye sahiptir:

byte < short < int < long < float < double < decimal

Daha küçük tipler daha büyük tiplere otomatik olarak dönüştürülebilir (Implicit Conversion), ancak büyük tipler küçük tiplere dönüştürülürken veri kaybı riski nedeniyle explicit casting (zorlama) gerektirir.

Bilişim Güvenliği Perspektifi: Sayısal Girdiler

Saldırganlar, programların sayısal sınırlarını zorlayarak (çok büyük veya çok küçük değerler girerek) buffer overflow veya integer overflow hatalarını tetiklemeye çalışabilirler.

Bu hatalar, programın bellek yapısının manipüle edilmesine ve zararlı kodun çalıştırılmasına zemin hazırlayabilir.

Giriş verilerinin daima beklenen aralıkta (bound checking) ve beklenen tipte olduğundan emin olmak (input validation) zorunludur.



Sonuç: Hesaplama Yeteneğinin Önemi

Bu hafta öğrenilen sayısal veri tipleri ve aritmetik operatörler, herhangi bir görsel programlama uygulamasının (masaüstü, web, mobil) temelini oluşturur.

Hesap makinesi projesi ile bu kavramları pratiğe döktük ve `.ToString()` metodunun veri akışındaki önemini anladık.

Gelecek hafta, `TextBox`'lardan alınan metin verilerini güvenli bir şekilde sayısal tiplere dönüştürme ve daha karmaşık formlar oluşturma üzerinde durulacaktır.

Sayısal Değişkenler ve Matematiksel Operatörler

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

